Final Project
Pipelined RV32-IMF CPU

Date: 2026/1/9

Presenter: Fang-Sheng, Lin

Outline

- Processor
  - IF and ID
  - EX
  - MEM and WB
- Instruction Cache
- Data Cache
- Minimal Clock Cycle Requirement

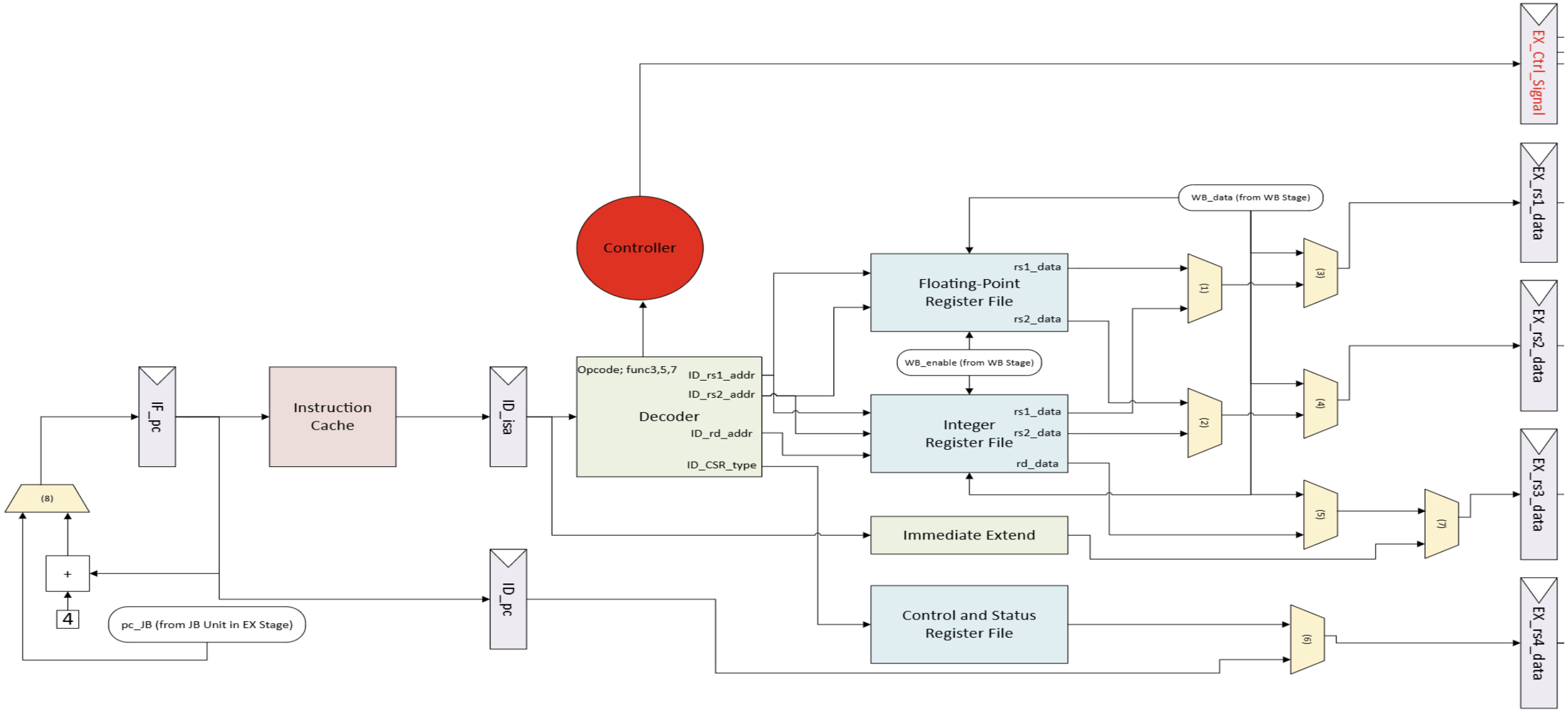
Processor

(1) IF and ID Stage

(2) EX Stage

(3) MEM and WB Stage

IF and ID Stage (1/3)



IF and ID Stage (2/3)

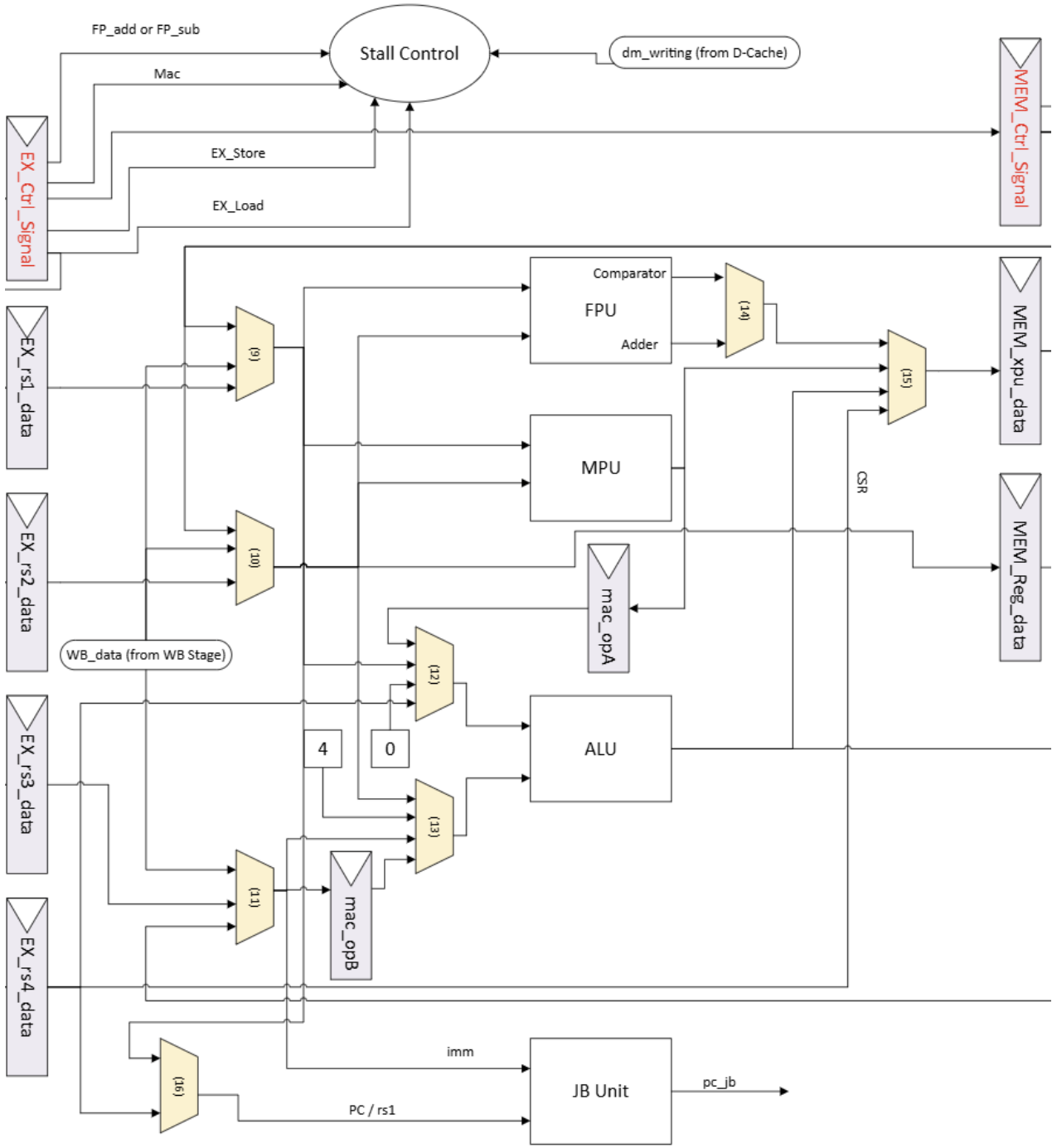
Multiplexer (Control by Controller)

(1)	RS1: Using Floating Point Register File or using Integer Register File
(2)	RS2: Using Floating Point Register File or using Integer Register File
(3)	RS1 Write-Back Forwarding
(4)	RS2 Write-Back Forwarding
(5)	Rd Write-Back Forwarding
(6)	CSR(only system) or PC
(7)	Rd(only Mac) or Imm
(8)	Select PC+4 or PC from JB Unit

IF and ID Stage (3/3)

- Adopt WB Forwarding
- Combine CSR and PC, imm and rd → Reduce 2 32-bit Registers
- ID needs to Stall
  - EX Stage stall
  - Load-Use Hazard → Stall current instruction 1 cycle and sent NOP to EX stage

EX Stage (1/5)


FP_add or FP_sub
Mac
Stall Control
dm_writing (from D-Cache)
MEM_Ctrl_Signal
EX_Ctrl_Signal
EX_Store
EX_Load
Comparator
FPU
Adder
(14)
(15)
MEM_xpu_data
EX_rs1_data
(9)
MPU
CSR
EX_rs2_data
(10)
mac_opA
MEM_Reg_data
WB_data (from WB Stage)
(12)
4
0
ALU
EX_rs3_data
(13)
(11)
mac_opB
EX_rs4_data
imm
(16)
JB Unit
pc_jb
PC / rs1

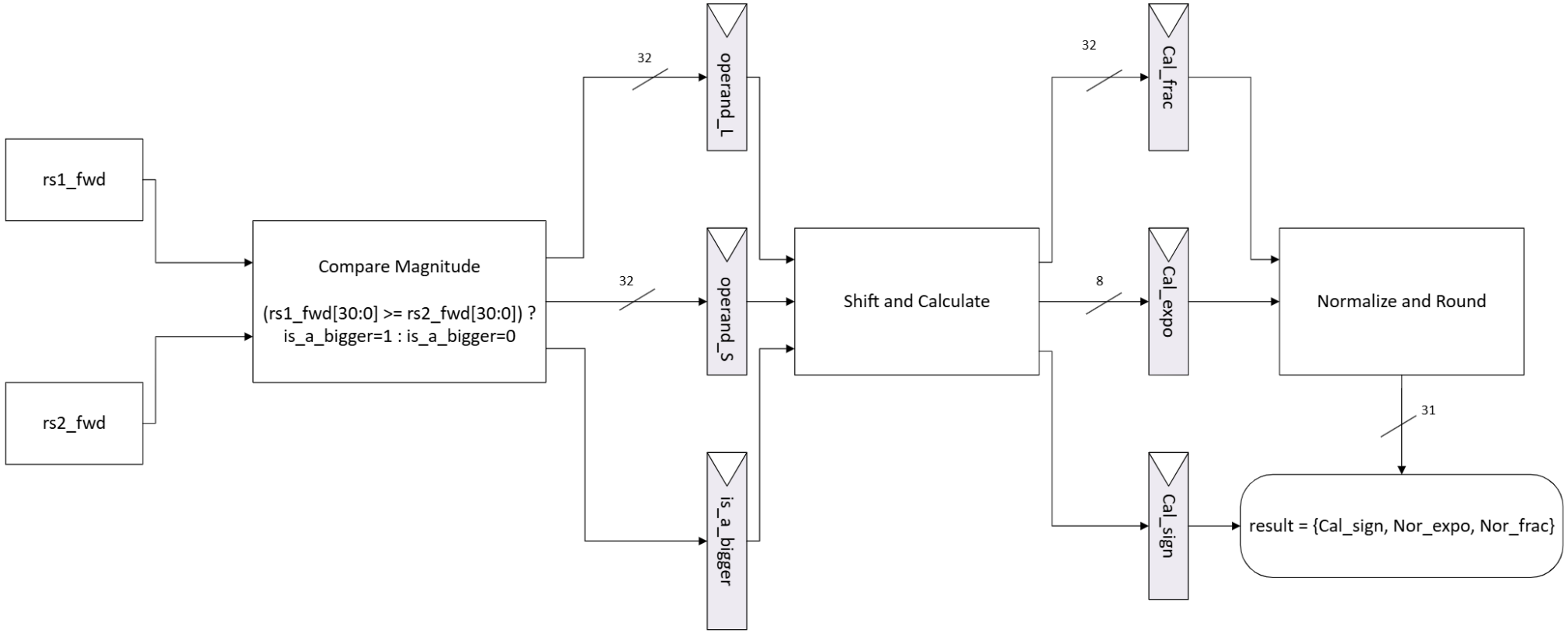
EX Stage (2/5)

Multiplexer (Control by EX_ctrl)

(9)	Rs1 Forwarding (from WB_data, MEM_xpu_data)
(10)	Rs2 Forwarding (from WB_data, MEM_xpu_data)
(11)	Rd Forwarding (from WB_data, MEM_xpu_data)
(12)	ALU operand 0 (rs1, pc, mac_opA, 32'd0)
(13)	ALU operand 1 (rs2, imm, mac_opB, 32'd4)
(14)	Result from FPU-Adder (F.add, F.sub) or FPU-Comparator (F.min, F.max)
(15)	CSR data (C&S instructions), FPU data, Multiplier data, ALU data
(16)	PC (Branch, JAL) or Rs1 (JALR)

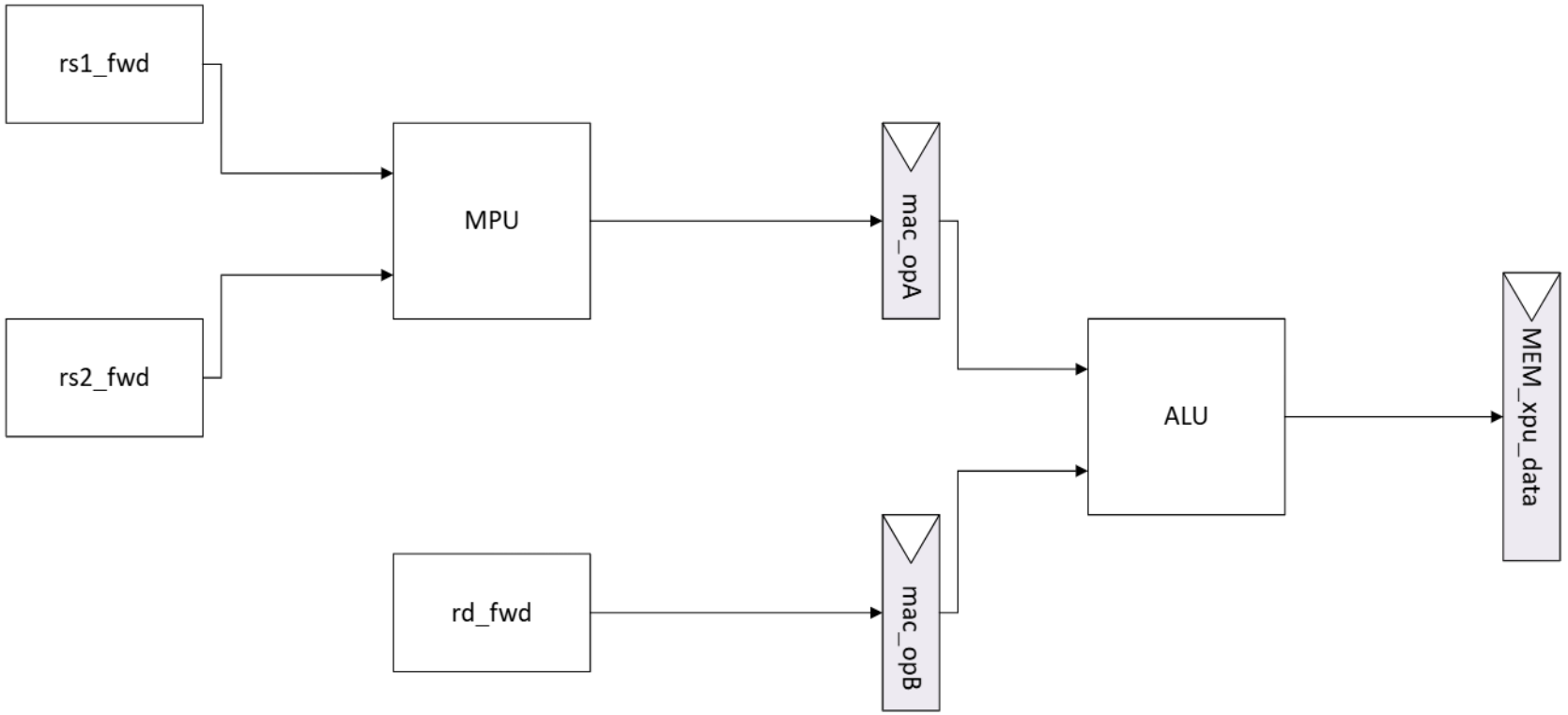
EX Stage (3/5)

- Floating-Point Adder (F.add and F.sub): 3 clock cycle



EX Stage (4/5)

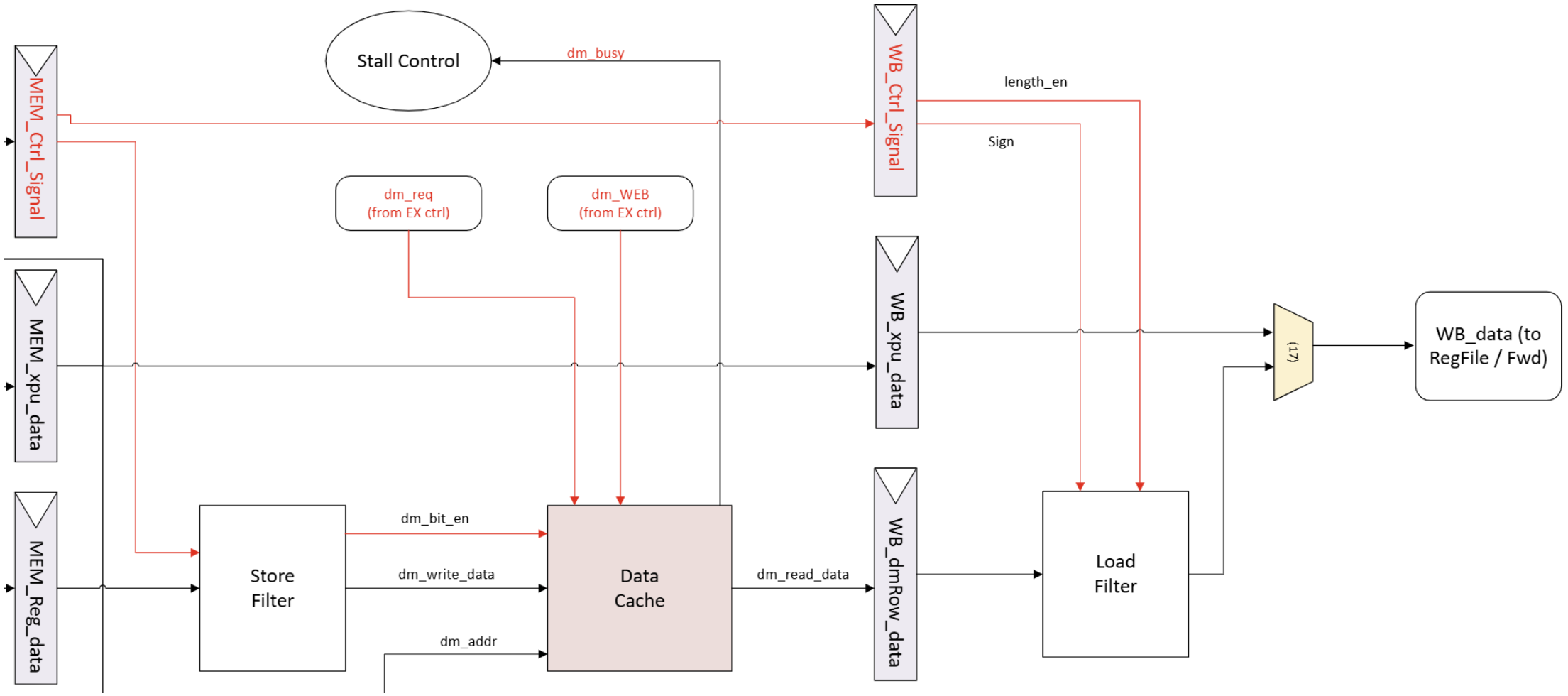
- Mac: 2 clock cycle



EX Stage (5/5)

- Connect rs1_fwd and rs2_fwd to MPU and FPU directly to skip one multiplexer’s passing time
- Adopt Multi-Cycle to reduce the critical path
- Adopt Forwarding
- Need to stall :
  - Mac (1 cycle)
  - F.add/F.sub (2 cycles)
  - DM still writing while Load/Store in this stage
  - MEM stage stall

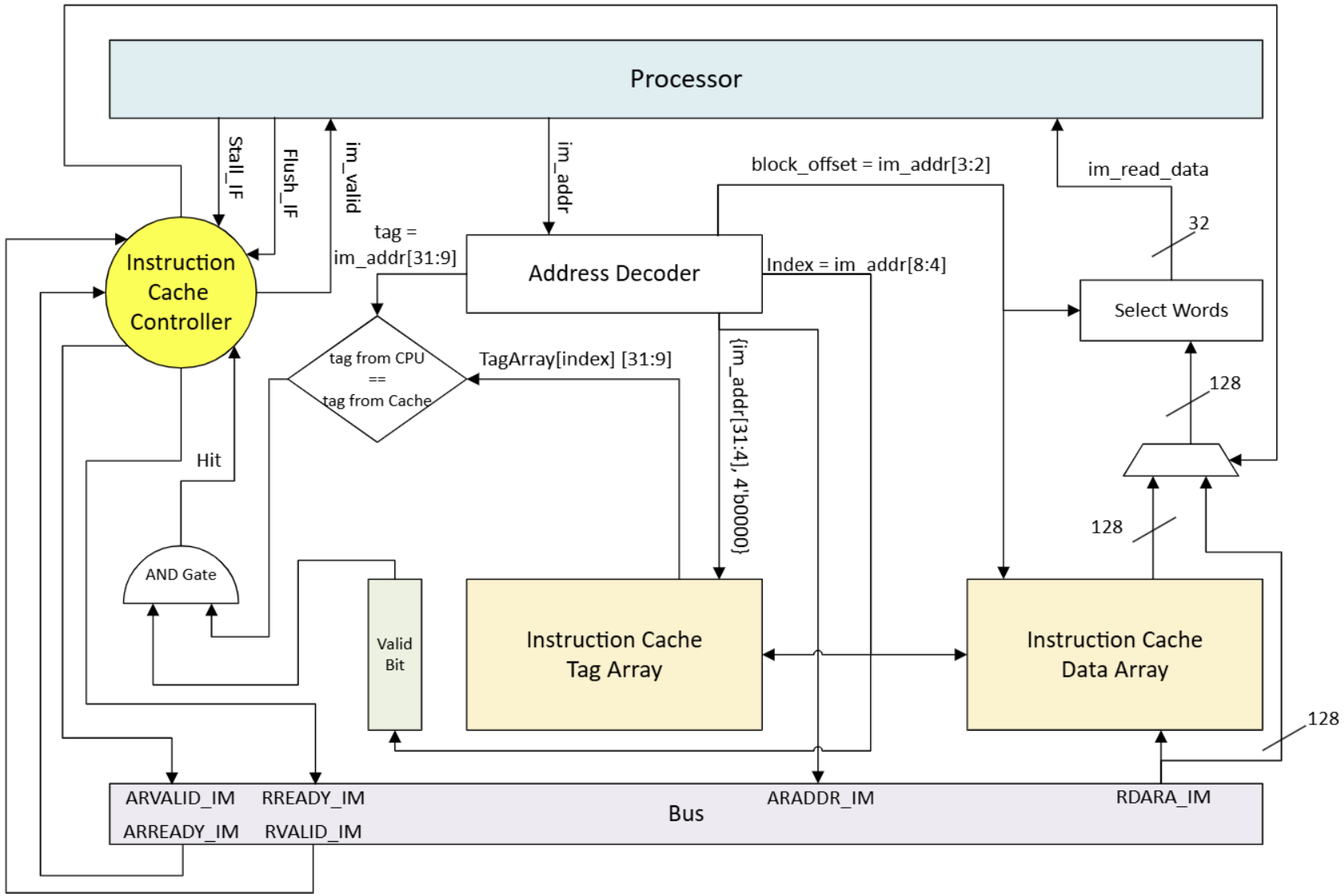
MEM/WB Stage



(17)	Data from DM (Load Instructions) or from Processor

Instruction Cache

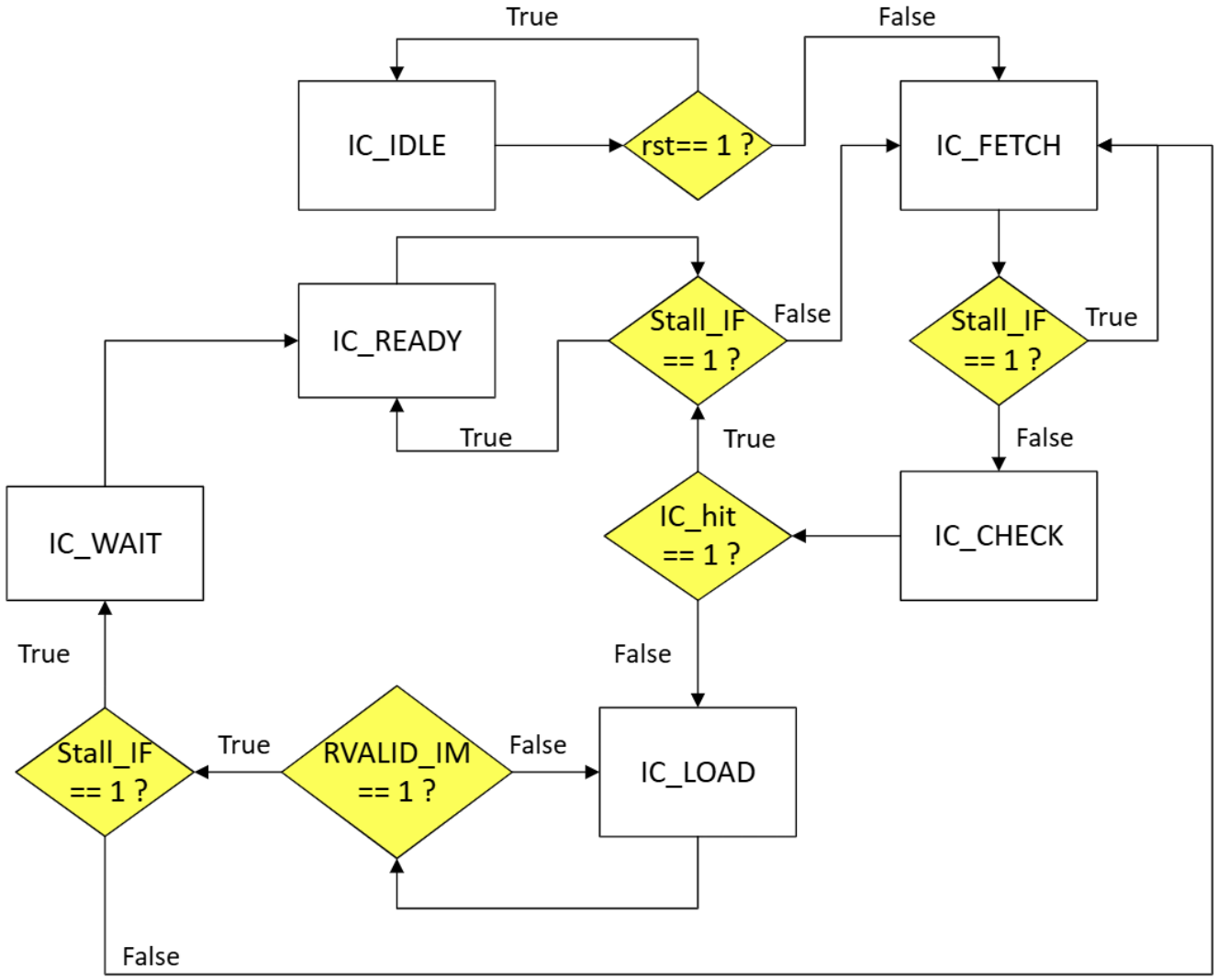
Instruction Cache (1/6)



Instruction Cache (2/6)

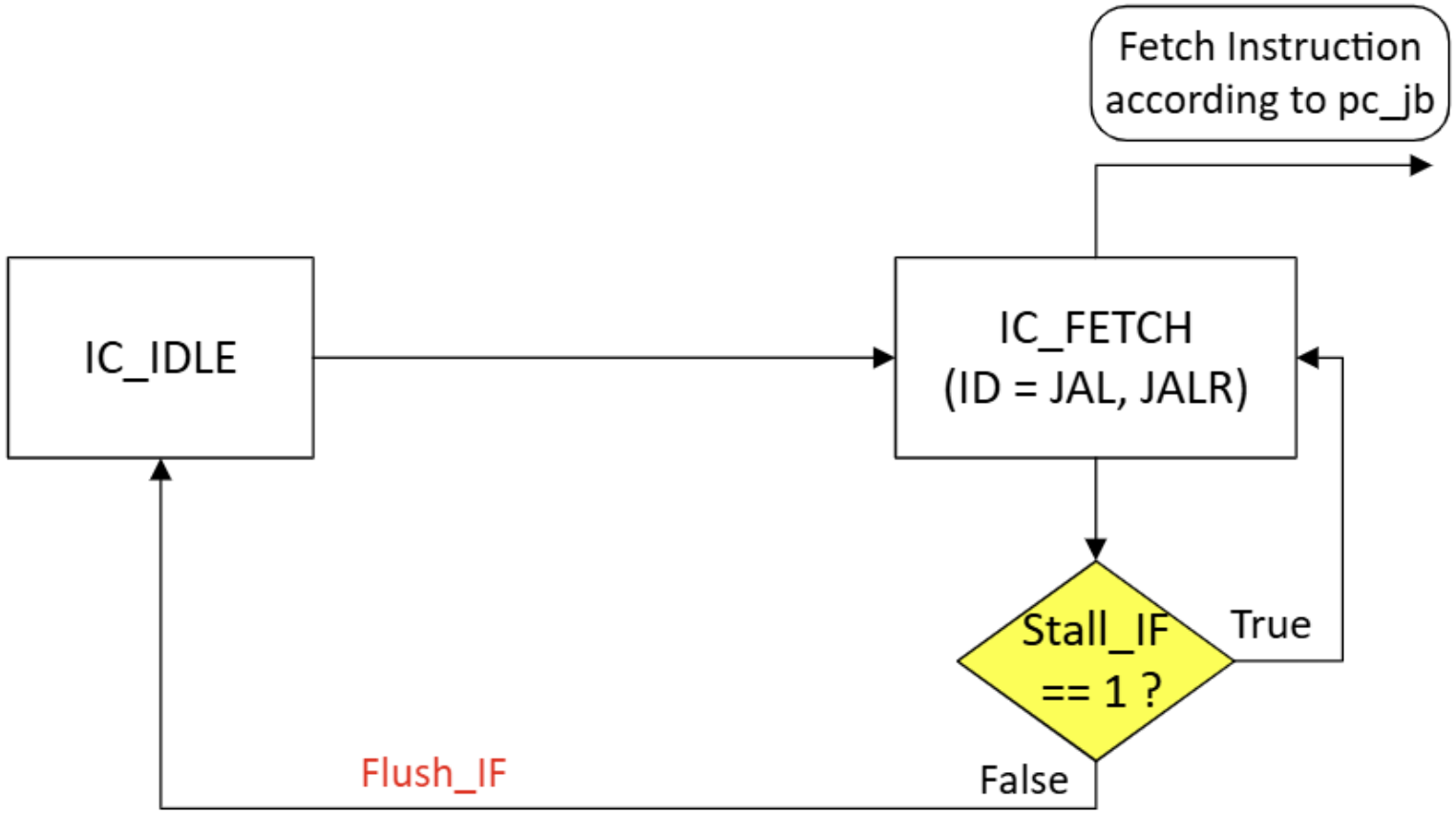
- Directed Mapped
- 512B
- Bypass
  - Hit: load from Cache
  - Miss: load from Bus (if ID not stalling when handshake)
- Finite State
  - IDLE
  - FETCH
  - CHECK
  - LOAD
  - WAIT
  - READY

Instruction Cache (3/6)



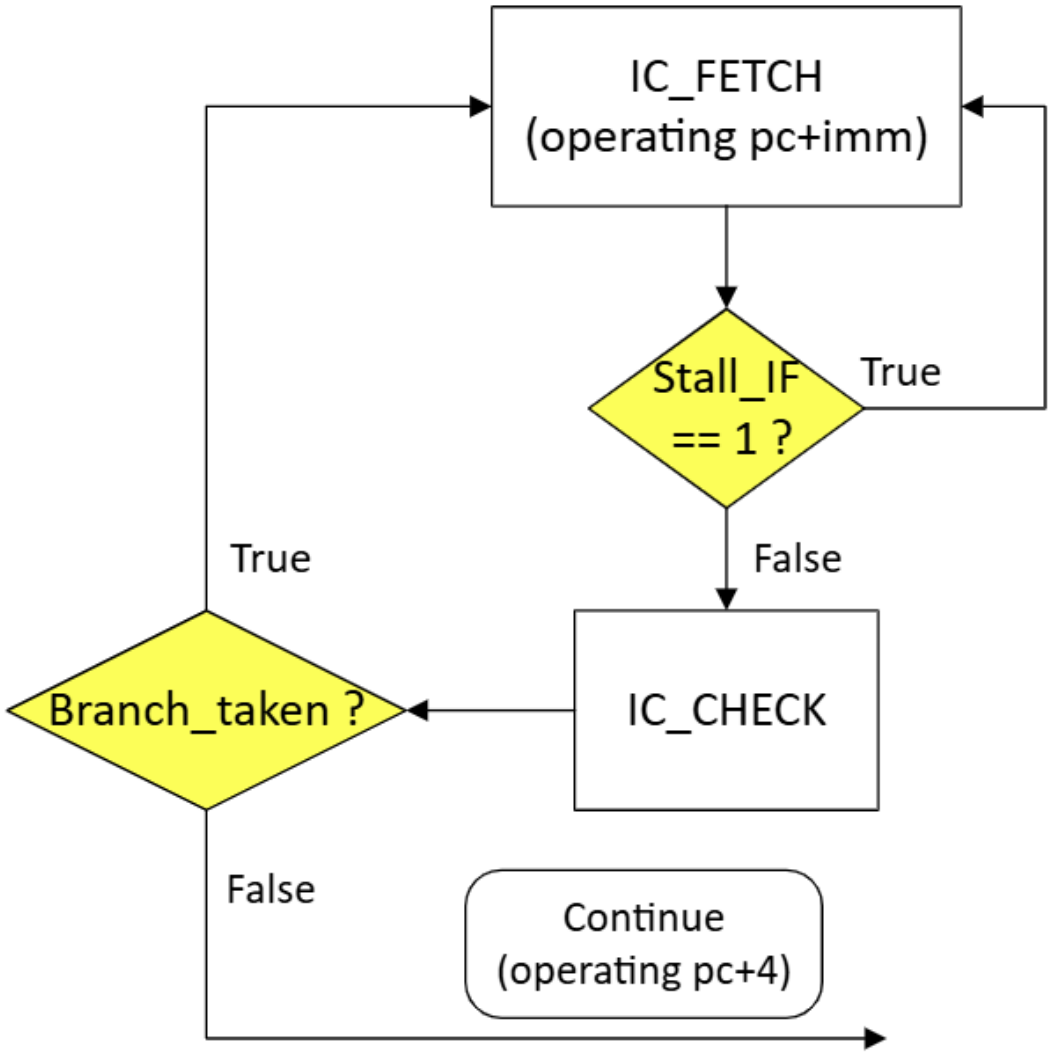
Instruction Cache (4/6)

Jump Instructions



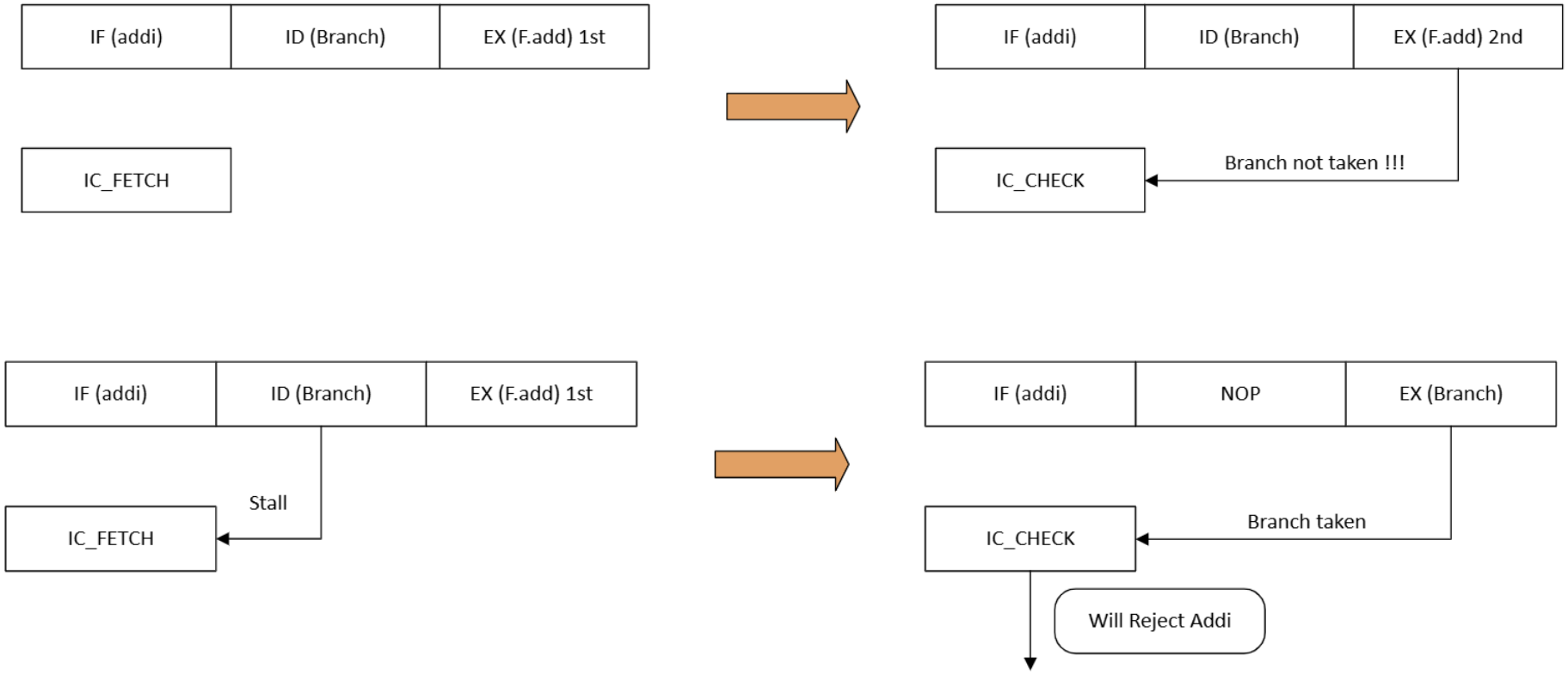
Instruction Cache (5/6)

Branch Instructions



Instruction Cache (6/6)

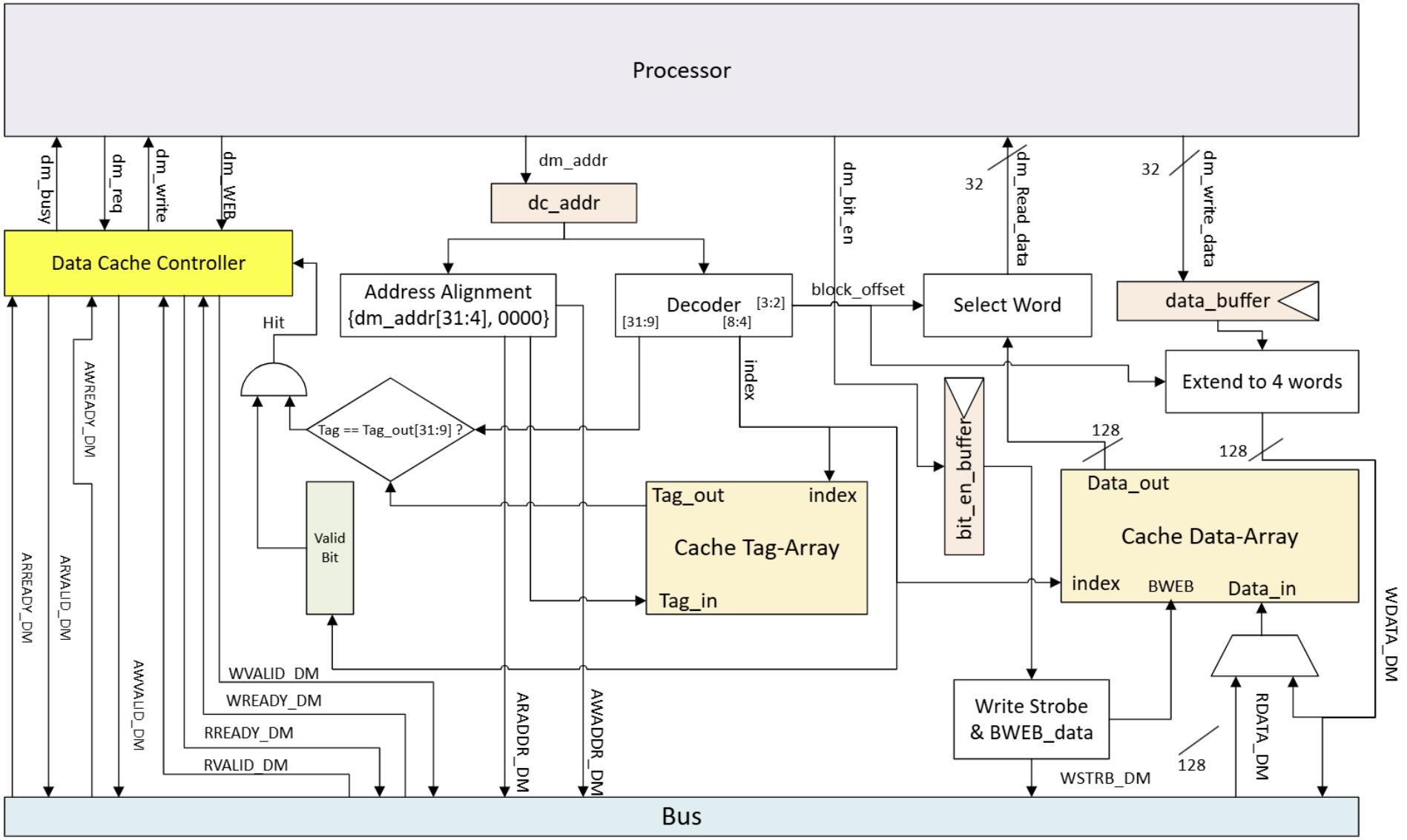
- Stall: If ID stall and IC_FETCH (start proceeding pc+4)



- Wait: if im_valid and Stall ID: need to hold and wait for ID

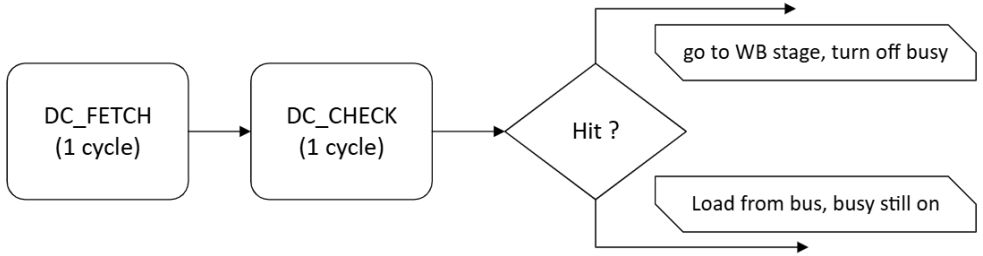
Data Cache

Data Cache (1/2)

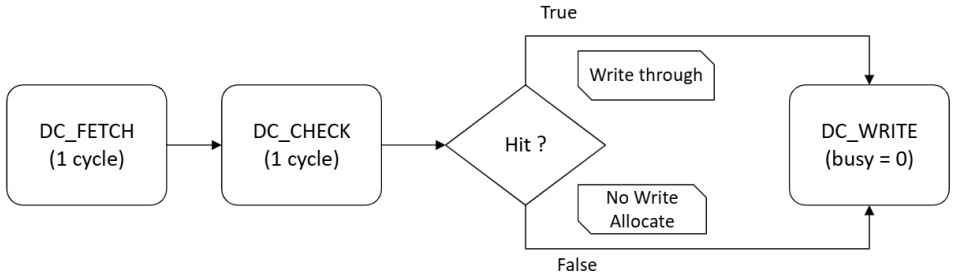


Data Cache (2/2)

- Directed-Mapped
- 512B
- Write Through + No Write Allocate
- Write Buffer
- Load: (2 + N cycles)



- Store: (2 + 1 cycles)



Minimal Clock Cycle Requirement

Minimal Clock Cycle Requirement

Ignore all kinds of stalls, assume Instruction Cache Hit

	IF	ID	EX	MEM	WB	Total
LUI	2	1	1	1	1	6
AUIPC	2	1	1	1	1	6
Jump	4	1	1	1	1	8
Branch	2(not taken) 4(taken)	1	1	x	x	4 6
Arithmetic R	2	1	1	1	1	6
Arithmetic I	2	1	1	1	1	6
Arithmetic F	2	1	3 (add, sub) 1 (min, max)	1	1	8 6
Mac	2	1	2	1	1	7
System	2	1	1	1	1	6
Store	2	1	1	2 + 1	x	6 + 1
Load	2	1	1	2 (Hit) 2 + N (Miss)	1	7 7 + N

The End